2026-08-14 · Research

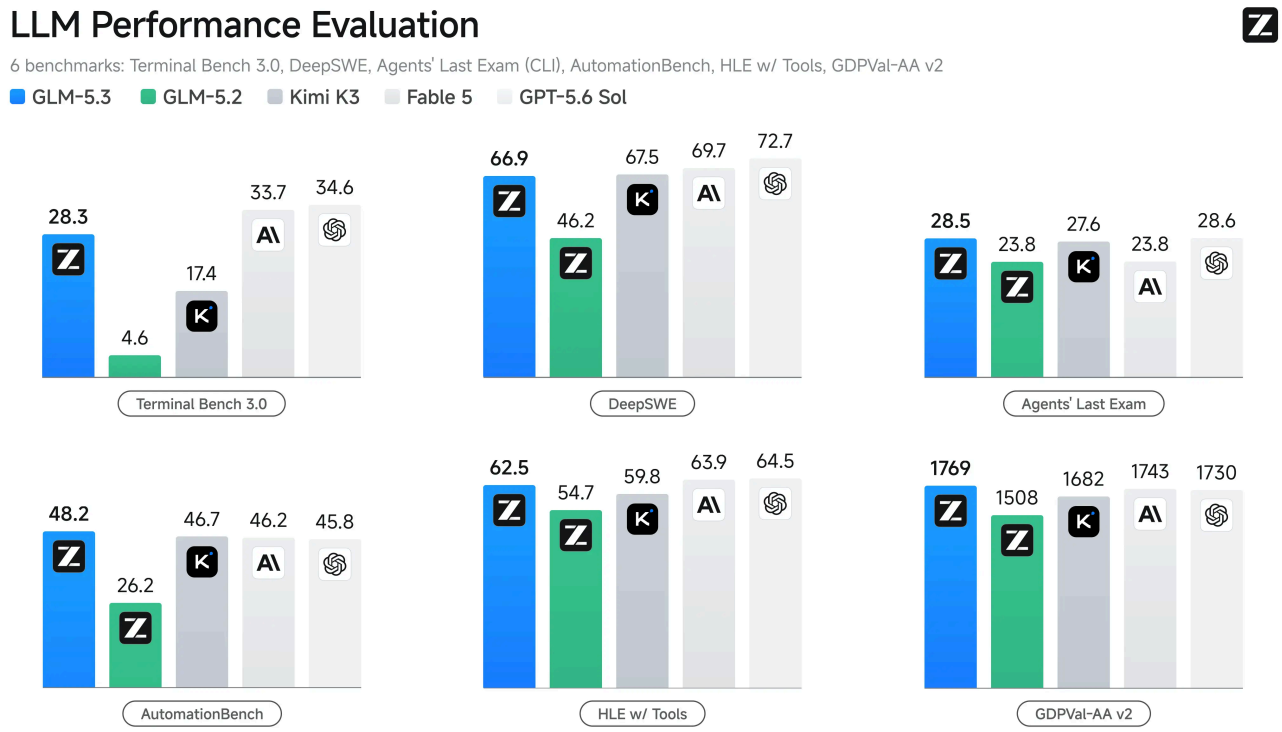
# GLM-5.3: Frontier Coding with Emergent Cyber Capabilities

Z.ai Coding Plan ↗ Code with ZCode ↗ HuggingFace (Coming Soon)

**Scaling post-training is all we did for GLM-5.3.** With GLM-5.2 we built the stack: IndexShare for efficient long-context processing, SAO for RL on long-horizon tasks, and slime for large-scale asynchronous training — all running on the long-horizon task environments we have been accumulating. Over the past month we kept scaling on this stack: more environments, more diverse tasks, and more compute spent training on them.

Today we are releasing GLM-5.3. It uses the same base model as GLM-5.2 — every gain comes from post-training. Compared with GLM-5.2, it is much better at complex coding and long-horizon tasks:

- **Stronger Coding:** GLM-5.3 is the most capable open-weights model for coding, with a 50% improvement over GLM-5.2 on our in-house Z.ai Code Bench. It also achieve open-source SOTA on public benchmarks including Terminal Bench 3.0 and Agents' Last Exam.
- **Emergent Cyber Capability:** As we scaled post-training, cyber capability developed faster than we expected. GLM-5.3 is state of the art on CyberGym for vulnerability discovery, and its gains are largest further up the exploitation chain, where it more than doubles GLM-5.2 on exploitation benchmarks.
- **Open Source:** We will release the weights in two weeks after launch, once safety evaluation and hardening are complete.




## Performance across comparison models

| Benchmark | GLM-5.3 | GLM-5.2 | Kimi K3 | DeepSeek-V4 Pro-0813 | Qwen3.8-Max | Opus 4.8 | Fable (w/ fallb |
|---|---|---|---|---|---|---|---|
| CODING | | | | | | | |
| Terminal Bench 2.1 | 88.2 | 81.0 | 88.3 | 87.9 | 86.6 | 85.0 | 88.( |
| Terminal Bench 3.0 | 28.3 | 4.6 | 17.4 | - | - | 21.1 | 33.7 |
| DeepSWE v1.1 | 66.9 | 46.2 | 67.5 | 62.7 | 56.6 | 58.0 | 69.7 |
| NL2Repo | 58.0 | 48.9 | 58.0 | 61.1 | 55.9 | 69.7 | - |
| ProgramBench Almost Solved | 19.0 | 9.5 | 17.5 | - | 10.5 | 15.5 | 33.( |
| FrontierSWE | 78.1 | 67.5 | - | - | - | 66.5 | 88.2 |
| SWE-Marathon v1.1 | 42.5 | 19.4 | 48.1 | - | - | 48.8 | 33.1 |
| PostTrainBench | 39.8 | 31.7 | 32.0 | - | - | 32.9 | 41.8 |
| CYBER | | | | | | | |
| CyberGym | 84.5 | 77.2 | 80.0 | 83.3 | 78.5 | 78.1 | 83.8 |
| ExploitGym 2h / 6h | 105 / 130 | 29 / 39 | 36 / 70 | - | 14 / 26 | 80 / 120 | 181 / 2 |

| Benchmark | GLM-5.3 | GLM-5.2 | Kimi K3 | DeepSeek-V4 Pro-0813 | Qwen3.8-Max | Opus 4.8 | Fable (w/ fallb |
|---|---|---|---|---|---|---|---|
| ExploitBench | 54.4 | 24.4 | 32.2 | - | 28.8 | 40.0 | 78.( |
| AGENTIC | | | | | | | |
| Toolathlon Verified | 73.0 | 59.9 | 76.5 | 74.1 | 72.5 | 76.2 | 74.7 |
| AutomationBench v1.0.6 | 48.2 | 26.2 | 46.7 | 43.2 | 39.8 | 41.0 | 46.2 |
| Agents' Last Exam ALE-CLI | 28.5 | 23.8 | 27.6 | 25.7 | 27.0 | 25.7 | 23.8 |
| HLE w/ Tools | 62.5 | 54.7 | 59.8 | 60.0 | 56.2 | 57.9 | 63.9 |
| GDPval-AA v2 | 1769 | 1508 | 1682 | 1590 | 1739 | 1588 | 174 |

# Stronger Coding

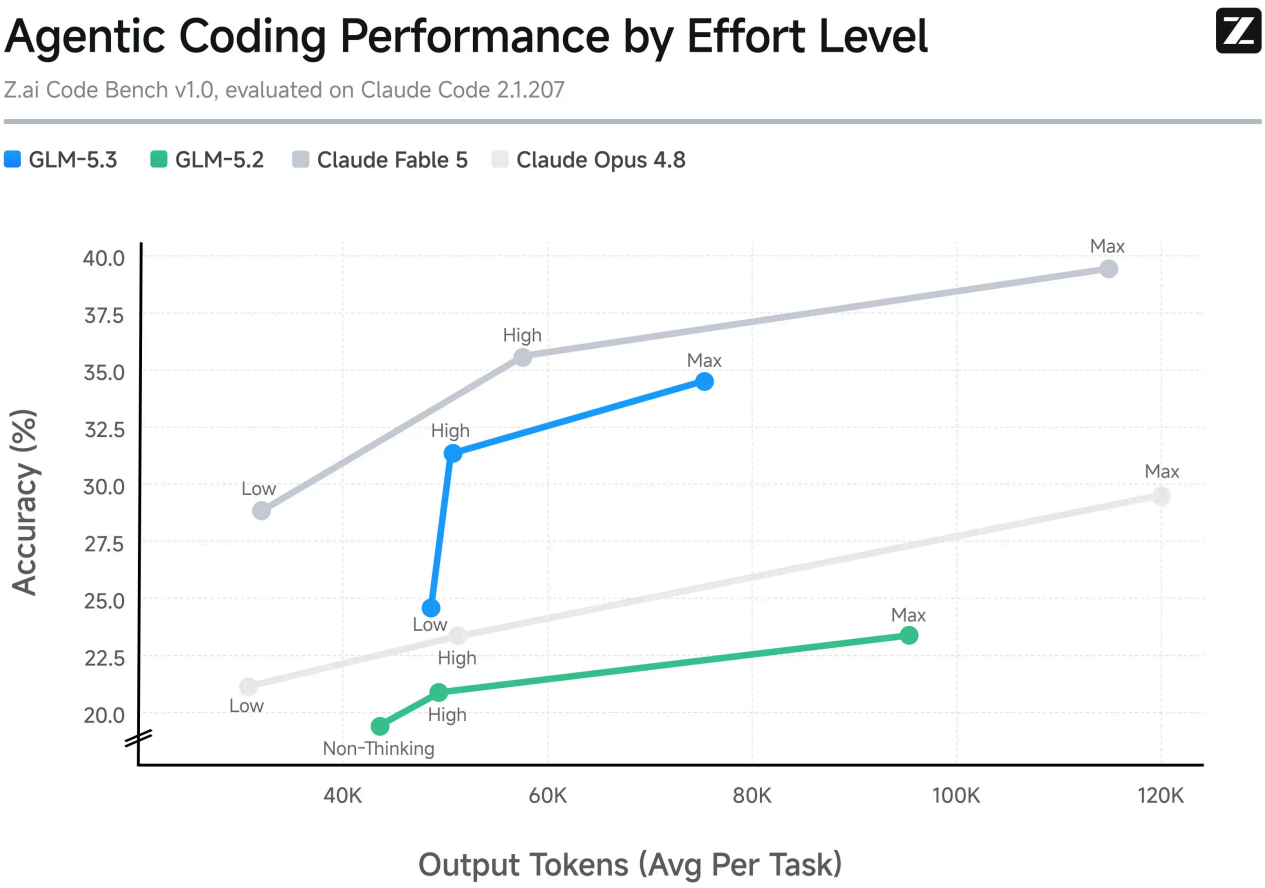
For GLM-5.3, we pushed environment scaling toward tasks that look less like coding exercises and more like real units of expert work. The environments now cover a much broader range of production workflows, with tasks designed around how engineering and research work is actually carried out in practice. Some represent several days of work for an experienced engineer. In an ML infrastructure task, for example, the model may be given the same working environment as an engineer, with access to compute clusters, storage systems, internal documentation, codebases, and experiment results. It must diagnose bottlenecks across the training stack, implement optimizations, run experiments, and deliver a measurable end-to-end speedup while preserving correctness. Training on environments at this level pushes the model toward taking ownership of substantial work end to end, rather than relying on users to decompose the problem and supervise each step.

As agent capability improves, much of the difficulty in scaling post-training moves from the model to the environment. A useful task environment has to be executable, verifiable, and close to real professional work — and we need many of them, not a handful of hand-built ones. To scale this process, we built pipelines that synthesize environments end to end, and for a subset of tasks, the RL reward signal as well. Research agents collect task patterns from real work and turn them into runnable long-horizon environments with multi-step dependencies and hidden state; a judge agent then attempts each task to verify that it is actually solvable. Verifiers are synthesized without access to the reference solution, while solver trajectories are

used to discover and close reward shortcuts. A verifier that passes oracle, no-op, and unsolved-state checks produces a binary reward reliable enough to train on directly.

It carries over the RL strategies introduced in GLM-5.2, including SAO with compaction, which helps these gains hold on long-horizon tasks rather than only on short ones. The effect shows up across both coding and general agent tasks. GLM-5.3 improves from 4.6 to 28.3 on Terminal-Bench 3.0, from 46.2 to 66.9 on DeepSWE v1.1, and from 23.8 to 28.5 on Agents' Last Exam. These pipelines still require a meaningful amount of human-in-the-loop work; making environment generation and verification more autonomous is one of the next steps.

Beyond public benchmarks, we introduce Z.ai Code Bench, an in-house benchmark designed to evaluate coding agents under realistic user scenarios. It covers diverse task categories and places agents in complex local development environments. At different effort levels, we evaluate agents along two dimensions: end-to-end task completion rate and fine-grained checklist accuracy. As a private benchmark, Z.ai Code Bench also reduces the risk of contamination from public test sets and gives us a more faithful measure of real-world user experience.

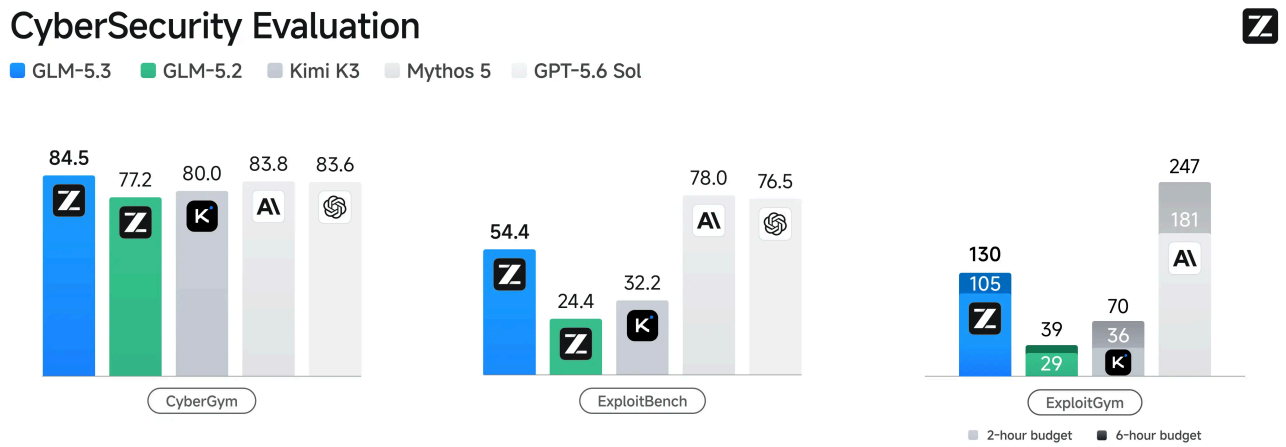



As shown in the figure, GLM-5.3 improves both performance and token efficiency. It delivers markedly stronger agentic coding results than GLM-5.2 at every effort level while consuming fewer output tokens. At Max effort, GLM-5.3 reaches 34.5% at roughly 75K output tokens per

task, compared with 23.4% at 96K for GLM-5.2. The same shift holds against closed models. At High effort, GLM-5.3 reaches 31.4% at around 50K output tokens, surpassing Claude Opus 4.8 at 29.5% with 120K. GLM-5.3 remains behind Claude Fable 5, which reaches 39.5% at Max effort.

# Emergent Cyber Capability

As part of post-training, we introduced vulnerability discovery data and environments into the training mix. We expected this to make the model better at finding and reasoning about vulnerabilities. What surprised us was how quickly the capability continued to develop as training scaled. GLM-5.3 did not simply become better at identifying isolated flaws: it began to reason across multiple stages of exploitation, forming coherent plans for complete exploitation chains.




We evaluate GLM-5.3 across three benchmarks covering different stages of vulnerability analysis and exploitation. On CyberGym, which starts from white-box source code and tests whether the model can identify and validate vulnerabilities by triggering faults, GLM-5.3 scores 84.5%, up from GLM-5.2's 77.2% — the best result on the benchmark, ahead of Mythos 5 (83.8%) and GPT-5.6 Sol (83.6%). On ExploitBench, which requires deeper reasoning about real vulnerabilities and their exploitation, GLM-5.3 reaches 54.4%, more than doubling GLM-5.2's 24.4%, while Mythos 5 and GPT-5.6 Sol score 78.0% and 76.5%, respectively. On ExploitGym, which measures how many exploitation tasks a model can complete under time-normalized budgets, GLM-5.3 completes 105 tasks within two hours and 130 within six hours, compared with 29 and 39 for GLM-5.2; budgets are normalized across models using per-model throughput figures, detailed in the footnotes. Mythos 5 remains well ahead at 181 and 247 tasks. The pattern across the three is consistent: the further up the exploitation chain a benchmark sits, the larger the gain from GLM-5.2 — and also the wider the remaining gap to the closed frontier. Capability is growing fastest exactly where we are furthest behind.

We then tested whether these capabilities transfer beyond controlled benchmarks. Since GLM-5.2, we have been working with several security teams in China to run our models against real-world codebases. After expert review, screening, and deduplication, the model identified 2,436 vulnerabilities across 269 projects, including 1,097 medium-to-high severity issues. The findings span system kernels, operating systems, browser engines, open-source infrastructure, web applications, and network protocols. Many had remained unnoticed for years or even decades, with the oldest dating back roughly 40 years.

This work has since grown into an ongoing disclosure effort. We built the Z.ai Security Disclosure Ledger to maintain a public record of the findings as they move through the disclosure process. The ledger is continuously updated as new vulnerabilities are reviewed and disclosed, distinguishing issues that have already been made public from those that are still under disclosure. For disclosed issues, it records information including the affected project, severity, CVE where available, and how long the vulnerability had remained in the codebase.

| 2,436 | 53 | 2,383 | 1,097 |
|---|---|---|---|
| FINDINGS TRACKED | PUBLICLY DISCLOSED | UNDER EMBARGO | CRITICAL & HIGH |
| **269** | **45** | | |
| OSS PROJECTS | YEARS OF IMPACT | | |

Findings span 45 years of impact - the oldest flaw was introduced in 1981, and on average a vulnerability lived 26.6 years before discovery.

### SEVERITY DISTRIBUTION

Critical 107 High 990 Medium 1,286 Low 53

### WHEN THE FLAWS WERE INTRODUCED

1981 2026

View Z.ai Security Disclosure Ledger ↗

# slime: Built for Long-Horizon RL Scaling

All of this runs on slime, our open-source post-training framework for RL scaling, with Megatron on the training side and SGLang on the rollout side. Its design keeps training, rollout, and the data buffer on a single dataflow, so math, code, sandboxes, verifiers, and long-horizon agentic environments plug in as data generation rather than as changes to the training loop. That is what let us keep adding environments through GLM-5.2 and GLM-5.3 without rebuilding the training stack each time.

Through GLM-5.3 we kept building it out on two fronts. On the algorithmic side we added capabilities aimed at RL research: top-p mask, top-k and full-vocabulary OPD, and configurations that improve training–rollout consistency, including R3-style setups and full numerical alignment between the training and rollout paths, which give us finer control over sampling, training, and teacher signals, and make it fast to run controlled comparisons. In our training–rollout consistency evaluation, the average difference in log probabilities (logprob) was controlled at the 1e-7 level, representing a reduction of more than 99.99% compared with previous setups.

We also worked on resource efficiency and system throughput for large-scale RL. Local storage now serves as an additional caching layer, holding model states and data hierarchically that would otherwise sit in host memory. This matters most for multi-teacher OPD: with dynamic teacher switching and prefetching on the training side, several teachers can be used without standing up a dedicated long-running inference service for each, at limited added overhead and substantially lower resource consumption. For agentic and asynchronous workloads, we improved joint scheduling and load balancing between the router and slime, so that rollout requests with widely varying lengths and completion times make better use of inference resources. We added workload-aware heuristics that derive throughput-oriented configurations — prefill/decode resource ratio, concurrency settings, and other throughput-critical parameters — from the characteristics of each rollout environment. As a result, for long-horizon coding RL tasks, these system-level optimizations improved end-to-end RL training throughput by more than 2.3×, allowing us to scale training over longer trajectories and more complex environments with substantially higher efficiency.

Taken together, these give us more experimental flexibility, lower resource cost, and higher throughput — which is what makes it practical to keep scaling RL.

# Getting started with GLM-5.3

# API Changes in GLM-5.3

GLM-5.3 supports three thinking effort levels: `low`, `high`, and `max`. Disabling thinking is no longer supported by GLM-5.3.

**Thinking Parameters**

| Parameter | Values | Default | Description |
|---|---|---|---|
| `thinking.type` | `enabled` | `enabled` | Enables thinking. `disabled` is no longer supported. |
| `reasoning_effort` | `low`, `high`, `max` | `max` | `low`: light; `high`: enhanced; `max`: deep. |

`max` is recommended for coding tasks.

```
{
  "model": "glm-5.3",
  "thinking": { "type": "enabled" },
  "reasoning_effort": "max"
}
```

**Migration required**: If your application currently uses `thinking.type: "disabled"`, change it to `enabled` and set `reasoning_effort` to `low` before updating the model ID to glm-5.3. **Otherwise, the request will fail.**

# Use GLM-5.3 with GLM Coding Plan & ZCode

Try **GLM-5.3** in your favorite coding agents—**ZCode, Claude Code, OpenCode**, and more. https://docs.z.ai/devpack/overview

**For GLM Coding Plan subscribers:** We've rolled out GLM-5.3 to all GLM Coding Plan users. The new GLM Coding Plan now uses a points-based quota system. Point usage is calculated separately for input, cached input, and output tokens. Model calls made outside peak hours consume 50% of the standard points. Peak hours are 14:00–18:00 (UTC+8), Monday through Friday; all other hours, including weekends, receive the 50% off-peak rate. Start building now: https://z.ai/subscribe

**Get more from GLM-5.3 with ZCode**

- 98%+ cache hit rate — repeated context billed at the lower cached rate, ~30% more effective tokens;
- 1.5x limited-time quota boost — stack it with the cache savings for up to 180% your standard quota through August 31.
- Long-horizon mastery — Goal mode plans, codes, tests, and verifies until the target is met;
- Remote Control — monitor and steer long-running tasks from your phone via WeChat or Feishu.

Try ZCode: https://zcode.z.ai

## Serve GLM-5.3 Locally

The model weights of GLM-5.3 will be publicly available soon in two weeks.

# Footnotes

- **HLE w/ tools**: We use sampling parameters of `temperature=1.0` and `top_p=0.95` for evaluation, with a maximum generation length of `163,840` tokens. The evaluation is conducted with a maximum context length of `300,000` tokens, using a context management strategy. We use GPT-5.6-luna (medium) as the judge model.
- **NL2Repo**: We evaluated NL2Repo with temperature=1.0, top_p=1.0, and max_new_tokens=64k under 1M context. To prevent hacking, we use rule-based and a LLM-based judgement to prevent malicious behaviors (e.g., unauthorized pip or curl operations).
- **DeepSWE**: We run DeepSWE using the mini-swe-agent harness with `temperature=0.95`, `top_p=1.0`, `timeout=6h` and 400K context.
- **Terminal-Bench 2.1**: We evaluate in Claude Code 2.1.207 with temperature=1.0, top_p=1, max_new_tokens=65536 with 6h timeout.
- **Terminal-Bench 3.0**: We evaluate Terminal-Bench-3 tasks with the Claude Code 2.1.207 harness (reasoning effort=max, 400K context, and 128K maximum output), reporting avg@3 over three rollouts per task. Each rollout runs in an isolated container built from the task's official image, and is capped at 600 agent turns with a 10-hour timeout. Tool Search is disabled, and the artifacts each agent produces are scored by the task's official separate verifier.
- **Agent's Last Exam (CLI)**: We evaluate ALE using the official evaluation protocol with the Claude Code harness (reasoning effort=max, 1M context, and 64K maximum output). Each of the 105 tasks runs in an isolated Docker container using the resources declared in its Task Card. The default timeout is 4 hours, with task-specific limits taking precedence (up to 8 hours). Tool Search is disabled, and results are scored by the official ALE evaluators.
- **Toolathlon Verified**: We obtain all results via the official evaluation service and report pass@1 averaged over 3 independent runs.
- **AutomationBench**: We evaluate on AutomationBench **v1.0.6**, incorporating the fix for the `null`-type handling issue introduced in PR #13.

- **GDPval-AA v2**: Models are evaluated by Artificial Analysis.
- **CyberGym**: We evaluate GLM-5.3 in Claude Code 2.1.207 (max reasoning effort, no web tools with temperature=1.0, top_p=1.0, max_new_tokens=128000). All evaluations are under unlimited timeout per task and results are single-run Pass@1 over 1,507 tasks. To simulate real-world usage scenarios, we place the agent inside the task container. We also remove all Git-related information and apply a domain whitelist (allowing only essential domains such as pypi.org and deb.debian.org for basic tool installation) to prevent the agent from cheating.
- **ExploitGym**: We evaluate GLM-5.3, Kimi-K3 and Qwen3.8 Max in Claude Code 2.1.207 (max reasoning effort, no web tools with temperature=1.0, top_p=1.0, max_new_tokens=128000). The reported results are single-run Pass@1 on 869 tasks under two timeout budgets: 2 hours and 6 hours, which are calculated as the API inference time rescaled by per-model tokens per second rate (per-model TPS sourced from Artificial Analysis; that is, we rescale GLM-5.3's results by 115 TPS, Kimi K3's results by 40 TPS and Qwen3.8 Max's results by 47 TPS), plus the non-API overhead. We also apply a domain whitelist (allowing only essential domains such as pypi.org and deb.debian.org for basic tool installation) to prevent the agent from cheating.
- **ExploitBench**: We evaluate GLM-5.3 in Claude Code 2.1.207 (max reasoning effort, no web tools with temperature=1.0, top_p=1.0, max_new_tokens=128000). Following the official evaluation settings, we limit the maximum number of interaction rounds between the agent and the environment to 300, and compute the average coverage score over all 41 tasks across 3 revisions. The coverage result of a task is determined by taking the union of capabilities achieved across all revisions, and the average score is obtained by averaging the results. We also apply a domain whitelist (allowing only essential domains such as pypi.org and deb.debian.org for basic tool installation) to prevent the agent from cheating.
- **FrontierSWE**: The evaluation was conducted by Proximal with 1M context length, max effort level, and 128K maximum output tokens. Dominance score reported as of 2026/08/14.
- **PostTrainBench**: We evaluate GLM-5.3 using Claude Code 2.1.207 with max effort level, temperature = 1.0, top_p = 1.0, max_new_tokens = 128000, and a 1M-token context window. We report the weighted average over 3 runs. Runs that fail to produce a score fall back to the official zero-shot base-model baseline score. For checks intended to prevent the use of third-party APIs, we removed the original pattern-matching-based checks, as they produced false positives when a local vLLM endpoint was accessed through the OpenAI SDK. Instead, we use an LLM agent to inspect solutions for external API usage.
- **SWE-Marathon**: We evaluate GLM-5.3 using Claude Code 2.1.207 with maximum effort level, temperature = 1.0, top_p = 0.95, max_new_tokens = 128000, and a 1M-token context window. For `strip-clone`, the original anti-cheat checks used overly broad import detection that could reject valid implementations. We removed the affected checks and performed llm-based inspection instead to avoid false positives. For `parameter-golf` and `trimul-cuda`, changes to the NVIDIA wheels caused the Docker image builds to fail, so we added `--extra-index-url https://pypi.org/simple` to restore successful builds.

**Legal**

Privacy Policy

Terms of Service